Lab 07 — Hands-on lab: web → api → db by name

Goal: see what the default rootless network really provides, create a network with DNS, publish ports, assemble the nginx → API → PostgreSQL chain by name, break nginx's DNS to understand the `502`, then do the same thing again inside a pod.

Prerequisites — Labs 01 to 06 completed. `~/labo-docker/05/Dockerfile` and `Api.java` present (lab 05). Images `alpine`, `nginx:alpine`, `eclipse-temurin:21-jdk`, `eclipse-temurin:21-jre-alpine`; `postgres:16-alpine` gets pulled again.

Files provided (`files/`) - `nginx-proxy.conf` — nginx serves the front end and relays `/api/` to `http://api:8080/` . - `nginx-pod.conf` — the same towards `http://localhost:8080/` , for the pod step. - `index.html` — the "Angular" page that calls `/api/actuator/health` .

The `10.89.0.x` addresses below are those of a freshly created network; yours may differ by one or two.

Step 0 — Rebuilding the API

```
mkdir -p ~/labo-docker/07 && cd ~/labo-docker/07
cp <path-to-the-lab>/files/* .
podman build -q -t api:1.0 ~/labo-docker/05
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep api
```

Observe `localhost/api:1.0 209 MB` : the multi-stage build from lab 05, rebuilt in a few seconds thanks to the cache.

Step 1 — The default network: pasta

```
podman run -d --name solo alpine sleep 1d
podman inspect solo --format '{{.NetworkSettings.IPAddress}}|{{.HostConfig.NetworkMode}}'
podman exec solo ip -4 -o addr show eth0
ip -4 -o addr show scope global | head -1
podman exec solo cat /etc/resolv.conf
podman exec solo wget -qO- --timeout=5 http://1.1.1.1 | head -c 60 ; echo
podman rm -f -t 0 solo
```

Observe `|pasta` — no address, `pasta` mode —, then an `eth0` inside the container carrying **the same address as your WSL** (`172.2x.x.x/20`), the host's `resolv.conf` , and some HTML: the Internet works.

Explanation. Without root privileges, no bridge: `pasta` reproduces the host's network stack inside the container. Handy for a lone container; useless for making several of them talk.

Step 2 — The `podman` bridge: IPs, no names

```
podman network ls
podman network inspect podman | grep -E '"(subnet|gateway|dns_enabled)"'
podman run -d --name p1 --network podman alpine sleep 1d
podman run -d --name p2 --network podman alpine sleep 1d
podman inspect p1 --format '{{.NetworkSettings.Networks.podman.IPAddress}}'
podman exec p2 ping -c1 -W1 $(podman inspect p1 --format
'{{.NetworkSettings.Networks.podman.IPAddress}}') | head -2
podman exec p2 ping -c1 -W1 p1
podman rm -f -t 0 p1 p2
```

Observe `"subnet": "10.88.0.0/16"`, `"dns_enabled": false`; `p1` has a `10.88.0.x` address; `ping` by IP answers (`64 bytes from 10.88.0.x`); `ping p1` fails: `ping: bad address 'p1'`.

Explanation. The bridge links the containers, but the `podman` network has no DNS. Hard-coding `10.88.0.4` in a configuration would be a time bomb: the address changes on the next creation.

Step 3 — A user-defined network with DNS

```
podman network create stack
podman network inspect stack | grep -E '"(name|subnet|gateway|dns_enabled)"'
podman run -d --name db --network stack -e POSTGRES_PASSWORD=secret postgres:16-alpine
podman run -d --name api --network stack -e APP_MESSAGE="hello from the stack network" api:1.0
sleep 3 ; podman exec api cat /etc/resolv.conf
podman run --rm --network stack alpine sh -c 'getent hosts api db; wget -qO-
http://api:8080/actuator/health; echo; nc -zv db 5432'
podman run --rm --network stack -e PGPASSWORD=secret postgres:16-alpine psql -h db -U postgres
-c 'select version()' | head -3
```

Observe `"subnet": "10.89.0.0/24"`, `"dns_enabled": true`; inside `api`, `search dns.podman` and `nameserver 10.89.0.1`; then `10.89.0.3 api.dns.podman … api`, `10.89.0.2 db.dns.podman … db`, `{"status":"UP"}`, `db (10.89.0.2:5432) open`, and `PostgreSQL 16.x on x86_64-pc-linux-musl`: a throwaway client reached the database **by name**.

```
podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'
```

Observe `api db`: the network knows its members.

Explanation. The network you create starts `aardvark-dns` on the gateway. Every container on the network resolves the names of the others. It is the only sane way to connect services.

Step 4 — Publishing a port

```
podman image inspect api:1.0 --format '{{.Config.ExposedPorts}}'
podman port api
curl -s --max-time 2 localhost:8080/actuator/health || echo "(unreachable from the host)"
```

Observe `map[8080/tcp:{}]` — the image *declares* 8080 —, an empty `podman port api`, and `(unreachable from the host)`: `EXPOSE` publishes nothing.

```
podman run -d --name api-pub --network stack -p 18080:8080 api:1.0
sleep 2 ; podman port api-pub ; curl -s localhost:18080/actuator/health ; echo
podman run -d --name api-rand --network stack -P api:1.0
podman port api-rand
podman run -d --name api-lo --network stack -p 127.0.0.1:18081:8080 api:1.0
podman port api-lo
podman ps --format 'table {{.Names}}\t{{.Ports}}'
```

Observe `8080/tcp -> 0.0.0.0:18080` and `{"status":"UP"}`; with `-P`, a random host port (`0.0.0.0:39843`); with `127.0.0.1:`, `127.0.0.1:18081`; and in `ps`, `db 5432/tcp` and `api 8080/tcp` **with no arrow** (exposed, not published) versus `api-pub 0.0.0.0:18080->8080/tcp`.

```
podman run -d --name api-80 --network stack -p 80:8080 api:1.0 ; echo "exit=$?"
podman ps -a --filter name=api-80 --format '{{.Status}}'
podman rm -f -t 0 api-pub api-rand api-lo api-80
```

Observe `Error: rootlessport cannot expose privileged port 80, you can add 'net.ipv4.ip_unprivileged_port_start=80' to /etc/sysctl.conf (currently 1024), or choose a larger port number (>= 1024): listen tcp 0.0.0.0:80: bind: permission denied`, `exit=126`, and a container left in `Created`.

Explanation. Publishing means asking the host to listen on the container's behalf. Under rootless, the host is you, and you do not have port 80.

Step 5 — The web → api → db chain

```
podman run -d --name web --network stack -p 18080:80 \
  -v ./nginx-proxy.conf:/etc/nginx/conf.d/default.conf:ro \
  -v ./index.html:/usr/share/nginx/html/index.html:ro nginx:alpine
sleep 1 ; curl -s localhost:18080/ | head -3
curl -s localhost:18080/api/actuator/health ; echo
curl -s localhost:18080/api/ ; echo
podman logs web 2>&1 | tail -1
```

Observe the front end's `<!doctype html>`, then `{"status":"UP"}` and `{"message":"hello from the stack network","profile":"default"}`: nginx relayed `/api/` to `api:8080`, stripping the prefix. In the log, the client address is `web`'s own (`10.89.0.10` here: the step 4 containers used up the previous addresses).

Explanation. A single published port, nginx's. The API and the database are reachable only by name, from the network. Open `http://localhost:18080` in your Windows browser: the page calls `/api/actuator/health` and displays `{"status":"UP"}` — no CORS, since everything comes from the same origin.

Step 6 — Breaking nginx's DNS

```
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'
podman stop -t 1 api
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman run -d --name intruder --network stack alpine sleep 1d
podman start api ; sleep 2
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman logs web 2>&1 | grep -i error | tail -1 | cut -c1-120
```

Observe `200` and `10.89.0.3`; after the stop, `504`; after the restart, the API has changed address (`10.89.0.12`: `intruder` took the previous one) and nginx answers `502`, with `connect() failed (113: Host is unreachable) while connecting to upstream`.

```
podman run --rm --network stack alpine wget -qO- http://api:8080/actuator/health ; echo
podman restart -t 1 web ; sleep 1
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman rm -f -t 0 intruder
```

Observe that the throwaway container reaches `api` without trouble (the DNS is up to date), and that after the `restart` of `web`, `200`.

Explanation. nginx freezes the IPs of its `proxy_pass` at startup. The DNS is not at fault: the client simply does not ask it again. Restart nginx, or give it a `resolver`.

Step 7 — Two networks and an alias

```
podman network create admin
podman run -d --name mon --network admin alpine sleep 1d
podman exec mon wget -qO- --timeout=2 http://api:8080/actuator/health || echo "api unreachable
from admin"
podman network connect --alias backend admin api
podman exec mon wget -qO- --timeout=2 http://backend:8080/actuator/health ; echo
podman inspect api --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}=
{{$v.IPAddress}} {{end}}'
podman network disconnect admin api
podman rm -f -t 0 mon ; podman network rm admin
```

Observe `wget: bad address 'api:8080'` then `api unreachable from admin`; after the connection, `{"status":"UP"}` through the `backend` alias; and two addresses: `admin=10.89.1.3 stack=10.89.0.12`.

Explanation. A network is also a boundary: `mon` could not see `api`. Connecting `api` to `admin` gives it an interface and a name on that network only.

Step 8 — host, none, and the host from a container

```
podman run --rm --network host alpine ip -4 -o addr show | grep -v ' lo '
podman run --rm --network host nginx:alpine sh -c 'nginx; sleep 1' 2>&1 | tail -1
podman run --rm --network none alpine wget -qO- --timeout=2 http://1.1.1.1 2>&1 | tail -1
podman run --rm --network none alpine ip -4 -o addr
podman run --rm alpine sh -c 'getent hosts host.containers.internal; wget -qO-
http://host.containers.internal:18080/api/actuator/health' ; echo
```

Observe your own WSL interfaces with `--network host`, then `nginx: [emerg] bind() to 0.0.0.0:80 failed (13: Permission denied)`; with `--network none`, `wget: can't connect to remote host (1.1.1.1): Network unreachable` and only `lo`; finally `169.254.1.2 host.containers.internal` and `{"status":"UP"}`: from the default network, the host and its published ports are reachable under that name.

Explanation. `host` = no isolation (and your user's limits); `none` = total isolation; `host.containers.internal` = "the machine I run on", Podman's equivalent of `host.docker.internal`.

Step 9 — The same chain inside a pod

```
podman pod create --name stackpod -p 18090:80
podman run -d --pod stackpod --name api-pod api:1.0
podman run -d --pod stackpod --name web-pod -v ./nginx-
pod.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine
sleep 2 ; podman pod ps
podman ps --pod --format 'table {{.Names}}\t{{.Pod}}\t{{.Ports}}'
curl -s localhost:18090/api/actuator/health ; echo
podman exec api-pod cat /etc/hosts | tail -2
podman pod rm -f -t 0 stackpod
```

Observe a `Running` pod with `3` containers (the third is the *infra*); in `ps --pod`, `api-pod` and `web-pod` share the same `POD ID` and the same publication `0.0.0.0:18090->80/tcp`, carried by `…-infra`; `{"status":"UP"}` through an nginx relaying to `localhost:8080`; and in `/etc/hosts`, `127.0.0.1 api-pod` and `127.0.0.1 web-pod`.

Explanation. Inside a pod, one network namespace for everyone: `localhost` links the containers, and the pod does the publishing. You have just run, without knowing it, the equivalent of a two-container Kubernetes Pod.

Clean-up

```
podman network rm stack
podman rm -f -t 0 web api db
podman network rm stack
podman network ls
podman rmi docker.io/library/postgres:16-alpine
```

Observe first `Error: "stack" has associated containers with it. Use -f to forcibly delete containers and pods: network is being used`, then, after removing the three containers by name, `stack` deleted and only `podman` left. Keep `api:1.0` and `~/labo-docker/07/`: labs 08 and 09 reuse the image and the nginx files.

What you must be able to state now

- Default rootless network = `pasta`: empty `IPAddress`, the host's address copied, no names.
- `podman` network: `10.88.0.x` IPs, `ping: bad address` by name; created network: `dns_enabled: true`, `getent hosts api` answers.
- `EXPOSE` publishes nothing: `podman port api` empty; `-p 18080:8080` publishes; `-P` picks a random port; `-p 80:…` → `permission denied` under rootless.
- nginx relays `/api/` to `api:8080` by name; one origin, no CORS.
- Recreated API = new IP = `502` at nginx, `200` after `restart web`.
- `--network host`: your interfaces and your limits; `--network none`: `Network unreachable`.
- A pod shares `localhost` and publishes by itself: `3` containers including the infra, `127.0.0.1 api-pod`.